

Admin Web Architecture & Operations Blueprint

Integrated UI/UX components, workflows, data contracts, security boundaries and five-app monorepo flow.

- ADMIN WEB
- CUSTOMER WEB
- CUSTOMER APP
- DRIVER APP
- MERCHANT WEB

Architecture principles

- 1

Firestore is the shared source of truth

All five apps read and write the same canonical records.
- 2

Admin Web is the operational control plane

Configuration, exceptions, imports, approvals and audits live here.
- 3

Share business logic, not forced UI

React web stays web-native; Expo apps stay mobile-native.
- 4

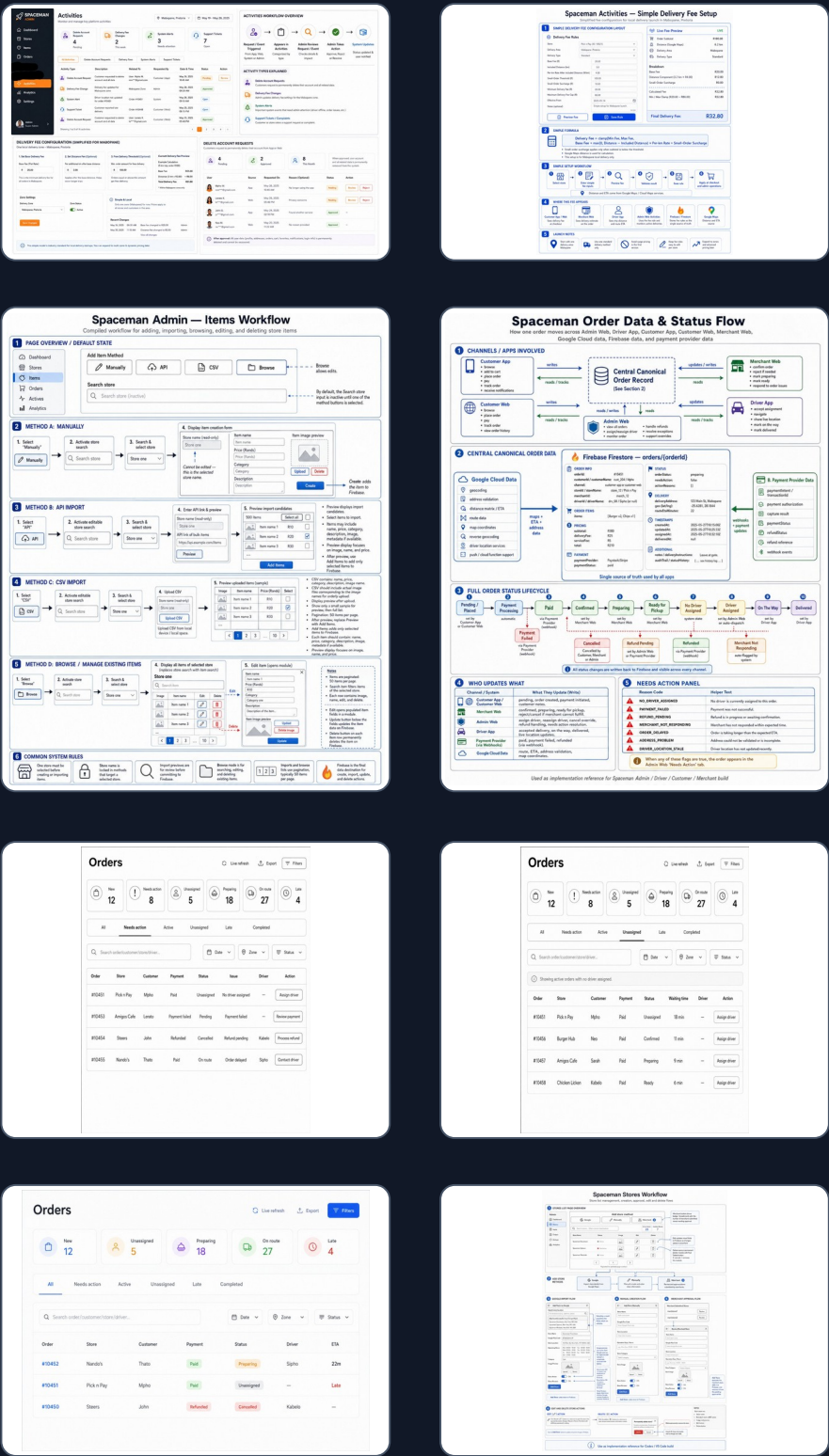
Privileged actions run server-side

Cloud Functions validate role, scope, idempotency and cleanup.
- 5

Codex changes are incremental and testable

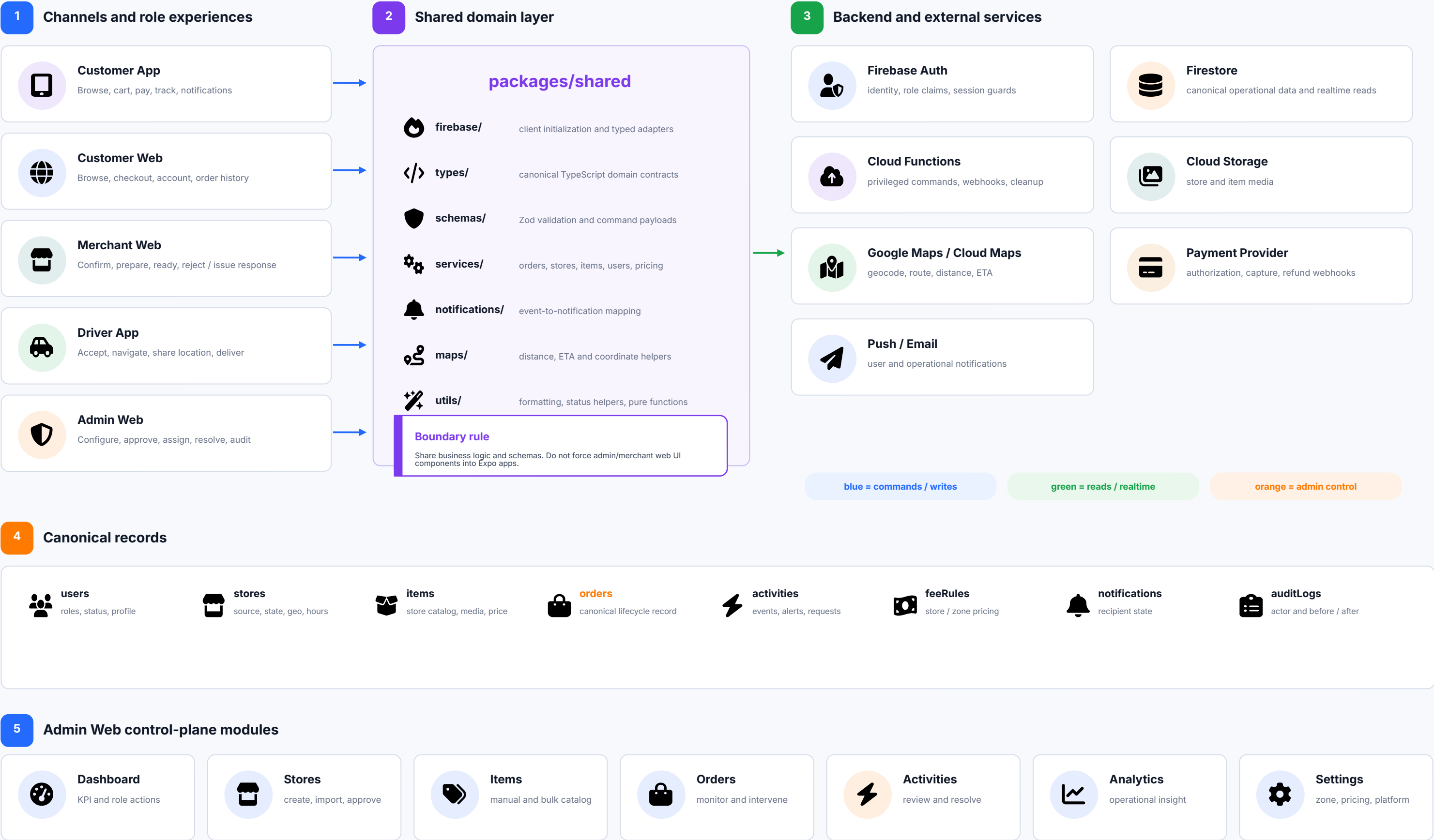
Inspect, plan, patch bounded scope, typecheck, build and report.

SOURCE BOARDS ANALYSED



Integrated Five-App System Blueprint

One Firebase backend, one canonical data model, five role-specific experiences and one Admin Web control plane.



Admin Web UI/UX Shell and Component System

Desktop-first operations interface with a collapsible sidebar, stable page anatomy and predictable modal/table behavior.

1

Page anatomy

SPACEMAN

ADMIN

Dashboard

Stores

Items

Orders

Activities

Analytics

Settings

Admin

Super Admin

Orders

Live refreshExportFilters

NewNeeds actionUnassignedPreparingOn route

12851827

AllNeeds actionActiveUnassignedLateCompleted

Search order / customer / store / driver...DateZoneStatus

Order	Store	Customer	Payment	Status	Driver	Action
#10451	Pick n Pay	Mpho	Paid	Unassigned	-	Assign
#10453	Amigos	Lerato	Failed	Pending	-	Review
#10455	Nando's	Thato	Paid	On route	Sipho	Contact

2

Reusable UX components

- 1

Navigation

collapsible sidebar, active-state highlight, account footer
- 2

Page header

title, optional location/date, live refresh, export, filters
- 3

KPI cards

label, count, icon, semantic color, click-through filter
- 4

Tabbed views

All / Needs action / Active / Unassigned / Late / Completed
- 5

Filter bar

search plus Date, Zone and Status selectors
- 6

Data tables

sortable headers, pagination, row actions, empty/loading/error states
- 7

Modals / drawers

create, edit, review, assign, refund, delete confirmation
- 8

Feedback

inline helper, validation, toast, confirmation and audit entry

Responsive rule

Admin Web is optimized for desktop/tablet operations. On narrow screens: collapse sidebar, stack KPI cards, move filters into a drawer and convert wide tables to horizontally scrollable data grids. Do not reuse this heavy layout in native apps.

Accessibility baseline

Keyboard focus, visible labels, 44 px targets, non-color status text, screen-reader names for icons and confirmation for destructive actions.

Dashboard, Account and Role-Management Workflows

Operational overview plus controlled creation of merchant, driver and admin identities.

1 Dashboard information hierarchy



Orders overview

- Confirmed orders - accepted by merchant and needing fulfillment
- Cancelled orders - investigate refund requirement
- Delivered orders - last 24 hours or selected period



Platform counts

- Stores
- Customers
- Admins
- Drivers
- Merchants



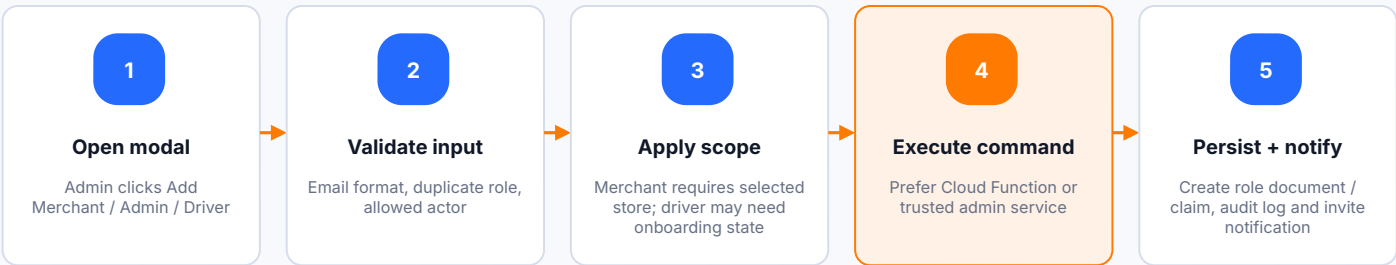
Primary actions

- Add Merchant
- Add Admin
- Add Driver
- Account
- Logout

Navigation behavior

The Dashboard "More" control routes to the Orders module. Counts may route to filtered lists. Sidebar toggle collapses labels to icons while retaining the active highlight.

2 Role creation command flow



3 Modal specifications and permissions

Add Merchant

email + store search / selection

Assign merchant to exactly the approved store scope. Cancel closes without side effects.

Add Admin

email

Super Admin only. Create role through privileged backend; never by client-only Firestore write.

Add Driver

email

Create driver identity / onboarding state; do not assign an order automatically.

Account Information

email, number, username

Logged-in admin may edit allowed profile fields. Role and claims are read-only here.

Logout

no form

Clear session and return to login. Invalidate local cached admin data.

User status model

Current project direction: administrators suspend / unsuspend users; customer deletion requests trigger archive / redact handling rather than destructive removal of financial and order history.

Audit requirement

Every role change records actorUid, targetUid/email, previousRole/status, newRole/status, storeScope, timestamp and command result.

Store Lifecycle - Create, Import, Approve, Edit and Retire

Manual creation remains first-class. Google and merchant submissions accelerate onboarding but never bypass admin review.

1 Stores list page

Add store method

Google

Manually

Merchant 3

Search stores...

Active 23

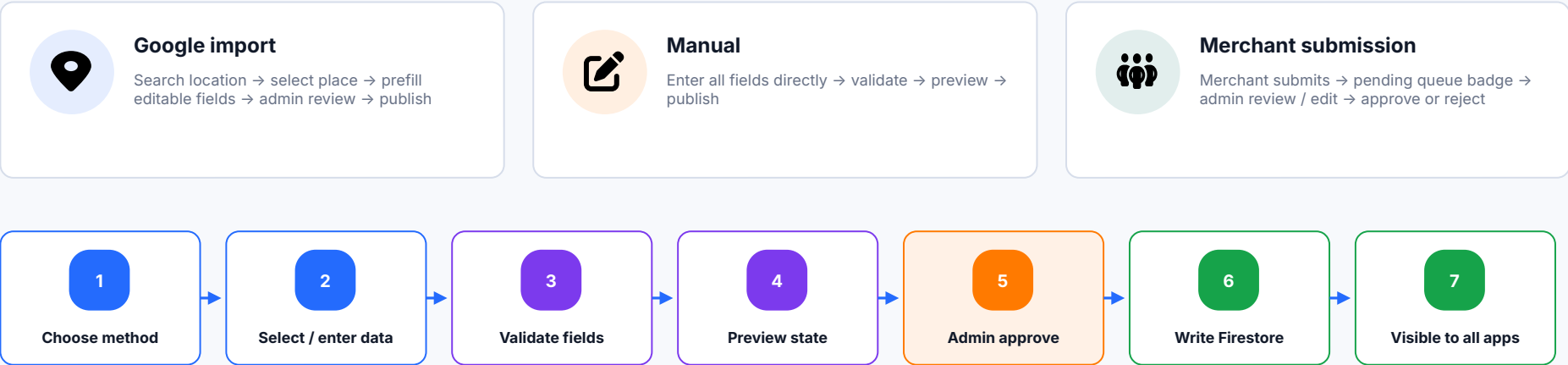
Inactive 7

Store name	Status	Image	Edit	Delete
Spaceman Downtown	Active	img	edit	trash
Spaceman Uptown	Not active	img	edit	trash
Spaceman Westside	Active	img	edit	trash

List behavior

Search filters the current store list. Pagination is server-backed. Status and image are visible; edit and delete are explicit row actions.

2 Onboarding routes



3 Store field contract

Identity	name, category, description, source
Location	address, Google Plus Code / placeld, geo point
Operations	opening hours, active / open-for-orders
Commerce	minimum order, delivery defaults, review visibility
Media	card image, hero image, attribution metadata
Governance	status: pending / active / inactive, createdBy, timestamps

4 Edit, delete and consistency rules

- Edit opens a populated form and writes changes to the shared store record.
- Manual, Google-imported and merchant-submitted stores are all editable by admin.
- Delete is a privileged command with confirmation; do not loosen Firestore rules.
- Production-safe delete preserves historical orders and removes/archives dependent catalog and favorites references.
- Prefer inactive / archived state for normal retirement; reserve hard delete for confirmed administrative cleanup.
- Create/update/delete must emit an activity and audit record.

Implementation optimization

Use a staging object for imports. Normalize place data once, attach sourceRef, avoid duplicates by placeld + normalized address, and never overwrite manually edited fields without admin confirmation.

Existing method must remain

Google import and merchant review sit beside the manual form; they do not replace or disable manual store creation.

Item Catalog Workflow - Manual, API, CSV and Browse

Every method targets a selected store, provides a review step and commits only validated data to the shared catalog.

1 Default state and method selection

Add Item Method

Manually

API

CSV

Browse

Search store (inactive until a method is selected)

Target lock

After store selection, the store name is read-only inside create/import/edit screens. This prevents cross-store item writes.

2 Four supported workflows



Manual create

1

Select method

2

Search/select store

3

Enter item name, price, category, description

4

Upload/delete image

5

Create



API import

1

Enter bulk API URL

2

Preview candidates

3

Select all or individual rows

4

Validate normalized values

5

Add selected items



CSV import

1

Upload CSV + image assets

2

Preview sample

3

Paginate (target 50/page)

4

Select rows

5

Add selected items



Browse/manage

1

Search selected store

2

Search item list

3

Edit populated modal

4

Update image/fields

5

Delete with confirmation

3 Validation and data-quality gate

Required

storeId, name, price, category

Price

numeric, non-negative, currency ZAR

Images

valid type/size, compressed upload, stable Storage URL

Duplicates

source ID + store ID + normalized name

Preview

show image, name and price before write

Selection

write only checked rows; no hidden bulk commit

Idempotency

re-running the same import must not duplicate items

Audit

source, actor, timestamp and import batch ID

4 Catalog data contract and performance

Core

itemId, storeId, name, description, category, price

Presentation

imageUrl, imageAlt, availability, sortOrder

Source

manual | api | csv, sourceRef, importBatchId

Governance

createdBy, updatedBy, createdAt, updatedAt

Browse performance

server pagination, default 50/page, indexed storeId + name search

Realtime policy

catalog uses cached queries; live listener only when operationally necessary

Media

compress images before upload; lazy-load thumbnails; delete orphaned media safely

Firestore rule

Firebase is the final destination for create, import, update and delete actions. Client UI never bypasses validation or role/store-scope checks.

Orders Operations Console

A single operational surface for monitoring, filtering, assigning and resolving orders without breaking the canonical lifecycle.

1

Information architecture

New12

Needs action8

Unassigned5

Preparing18

On route27

Late4

All

Needs action

Active

Unassigned

Late

Completed

Search order / customer / store / driver...

Date

Zone

Status

Filters

3

Needs-action reason to action mapping

NO_DRIVER_ASSIGNED→ Assign driver

REFUND_PENDING→ Process refund

ORDER_DELAYED→ Contact driver / reassign

DRIVER_LOCATION_STALE→ Contact driver / refresh location

PAYMENT_FAILED→ Review payment

MERCHANT_NOT_RESPONDING→ Contact merchant / resolve

ADDRESS_PROBLEM→ Review address

Top actions

Live refresh updates operational data; Export produces a filtered snapshot; Filters opens advanced constraints. Use debounced search.

2

Tab-specific table contracts

AllOrder, Store, Customer, Payment, Status, Driver, ETA

Needs actionOrder, Store, Customer, Payment, Status, Issue, Driver, Action

UnassignedOrder, Store, Customer, Payment, Status, Waiting time, Driver, Action

ActiveOrder, store, current stage, driver, ETA, Issue flag

LateETA breach, waiting time, contact / reassign action

Completeddelivered / cancelled / refunded with timestamps

4

Realtime, SLA and safe intervention rules

- Use one scoped realtime query per visible operational view; unsubscribe on tab / route exit.
- Derived flags are computed from canonical status, timestamps, ETA and payment state - not stored as uncontrolled UI state.
- Assign, cancel override and refund commands require confirmation, validation and audit.
- Row actions must be context-sensitive and disabled while a command is pending.
- Every update returns the refreshed canonical order and surfaces a toast / error.

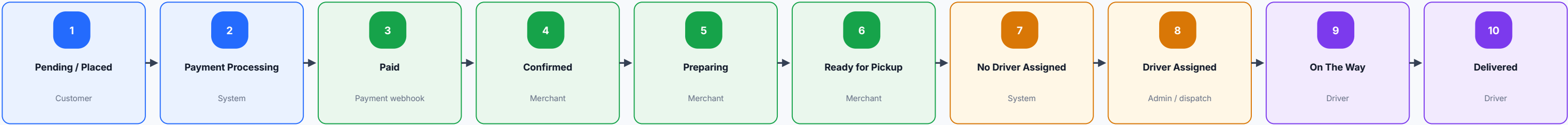
Visual state

Status badges always combine color and text: Paid, Refunded, Preparing, Unassigned, Cancelled, Late. Never rely on color alone.

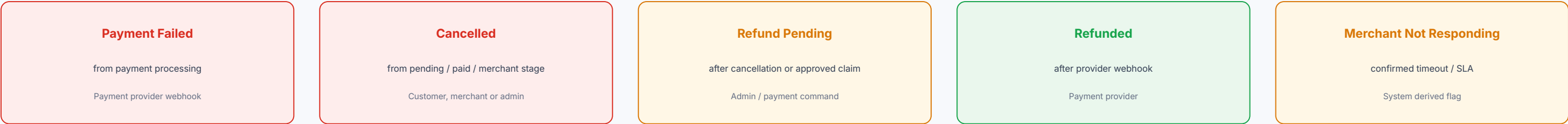
Canonical Order Lifecycle and Operational Ownership

All status changes are written back to the same order record and become visible across every channel.

1 Happy-path lifecycle



2 Exception branches



3 Who writes what

Customer App / Web	pending, order creation, payment initiated, customer notes
Merchant Web	confirmed, preparing, ready for pickup, reject / cannot fulfill
Admin Web	assign / reassign driver, cancel override, refund handling, needs-action resolution
Driver App	accepted assignment, on the way, delivered, live location
Payment Provider	paid, payment failed, refunded via signed webhook
Maps / Cloud Data	route, ETA, geocode and validated coordinates

4 Derived needs-action logic

NO_DRIVER_ASSIGNED	active order has no driver
PAYMENT_FAILED	provider reports unsuccessful payment
REFUND_PENDING	refund command not confirmed
MERCHANT_NOT_RESPONDING	merchant exceeds confirmation SLA
ORDER_DELAYED	current time exceeds expected ETA
ADDRESS_PROBLEM	validation failed or incomplete
DRIVER_LOCATION_STALE	location timestamp exceeds freshness threshold

Invariant

Status transitions are validated server-side. A UI cannot jump directly from Pending to Delivered, mark Paid without a provider event, or refund more than the captured amount.

Canonical Order Record and Cross-System Data Flow

The Firestore order document is the operational single source of truth; external providers enrich it through validated commands and webhooks.

2 Google Cloud / Maps enrichment

 geocoding

 address validation

 distance matrix / ETA

 route polyline

 map coordinates

 reverse geocoding

 driver location services

 push / Cloud Function support

Write pattern

Maps results are normalized by a service / function and written into delivery fields with source timestamp. Do not let each app invent its own ETA formula.

4 Cross-app read / write model



Customer App / Web
writes order creation and notes; reads tracking



Merchant Web
writes merchant statuses; reads assigned store orders



Driver App
writes location / delivery status; reads assigned orders



Admin Web
reads all scoped data; writes operational commands



Functions
validates privileged transitions and propagates activities / notifications

1 Firestore - orders/{orderId}

SINGLE SOURCE OF TRUTH

ORDER INFO

orderId, customerId / snapshot, channel, storeId / snapshot, merchantId, driverId / snapshot

ORDER ITEMS

item snapshots: id, name, quantity, unit price, modifiers

PRICING

subtotal, deliveryFee, serviceFee, discounts, total, currency

PAYMENT

provider, transaction / intent ID, paymentStatus, capture / refund references

STATUS

orderStatus, previousStatus, needsAction, actionReasons[]

DELIVERY

address snapshot, geo, routeEtaMinutes, distanceKm, instructions

TIMESTAMPS

createdAt, updatedAt, paidAt, assignedAt, pickedUpAt, deliveredAt

ADDITIONAL

notes, auditTrail / statusHistory references, schemaVersion

3 Payment provider integration

 paymentIntent / transactionId

 authorization result

 capture result

 paymentStatus

 refundStatus

 refundReference

 signed webhook events

Webhook rule

Verify signature, check idempotency key, compare amount/currency and persist event ID before mutating payment or refund status.

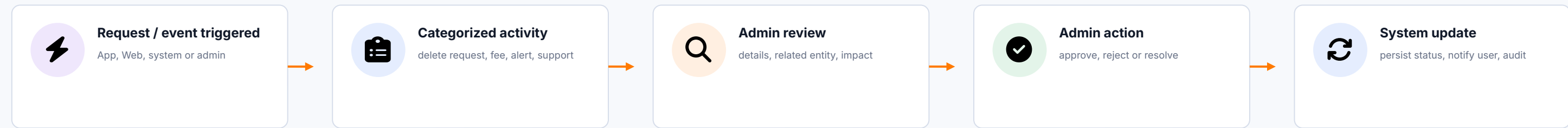
Snapshot principle

Store, customer and item display values are snapshotted into the order so later catalog/profile edits do not rewrite historical transaction meaning.

Activities, Delivery Fees, Support and Account Lifecycle

The Activities module converts events and requests into reviewable admin work with explicit outcomes and auditability.

1 Activity workflow



2 Activity types and table contract

1	Delete account requests	customer request, source, reason, status, review / reject
2	Delivery fee changes	rule, store/zone, previous and new values, actor
3	System alerts	driver offline, stale location, order issue, integration failure
4	Support tickets	customer / store issue, priority, owner, resolution

Table columns						
Activity type	Description	Related to	Requested by	Date & time	Status	Action. Tabs filter All, Delete Account, Delivery Fees, System Alerts and Support Tickets.

3 Delivery fee rule

Delivery Fee = clamp(Min Fee, Max Fee,

Base Fee + max(0, Distance - Included Distance) x Per-km Rate + Small-Order Surcharge)

Store / delivery area	Pick n Pay - Mabopane
Delivery type	Standard
Base fee	R20.00
Included distance	3.0 km
Per-km rate	R4.00
Small-order threshold / surcharge	R100 / R10
Minimum / maximum	R20 / R80
Effective from / notes	versioned rule metadata

Launch constraint

Start with one delivery area (Mabopane), one standard delivery type and no surge pricing. Distance and ETA come from Google Maps. Expand to zones and advanced pricing later.

4 Account deletion decision - current project rule

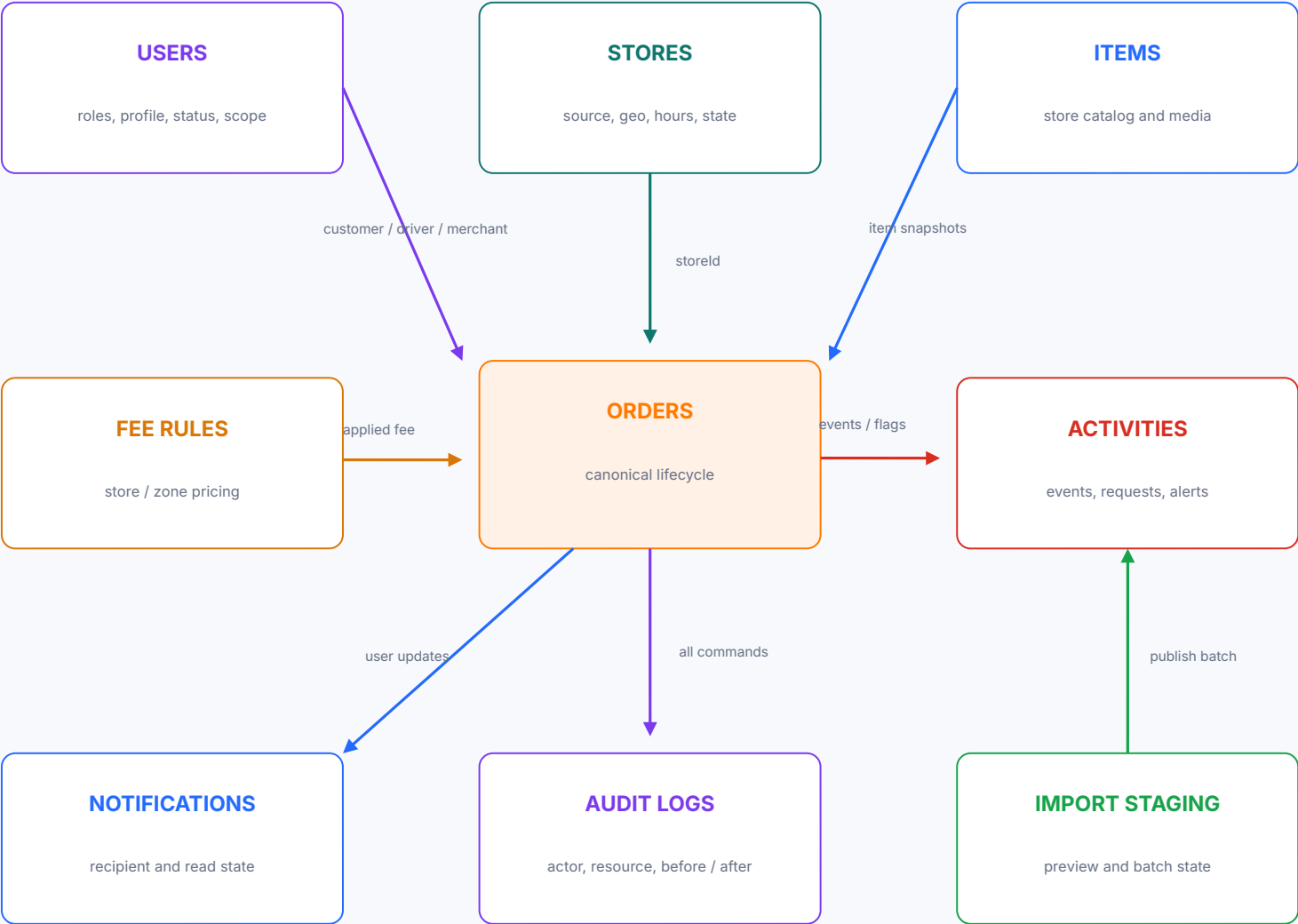
Supersedes the destructive mockup

On approval: disable sign-in, mark user archived, redact personal profile/contact data, remove disposable data (cart, favorites, notification tokens) and retain required order/payment/audit records with non-identifying references. Suspend / unsuspend remains available for temporary access control.

Logical Firestore Data Model and Ownership Contract

Only orders/{orderId} is explicitly fixed by the source diagrams; other paths below are the recommended logical contract to confirm against the current repo before implementation.

1 Entity relationship map



Historical integrity

Orders contain customer/store/item snapshots. References support navigation, but historical values must not be recomputed from mutable catalog/profile documents.

2 Recommended collection contract

users/{uid}	role, status, profile, storeScope	self + admin
stores/{storeId}	catalog owner, geo, source, state	admin + scoped merchant
items/{itemId}	storeId, price, media, availability	admin + scoped merchant
orders/{orderId}	canonical order data	channel owners by transition
activities/{activityId}	type, relatedTo, status, action	system/admin
feeRules/{ruleId}	store/zone pricing versions	admin only
notifications/{id}	recipient, event, read state	system + recipient
auditLogs/{id}	immutable command history	server only
importBatches/{id}	staging, preview, results	admin/import service

Schema discipline

Every document has schemaVersion, createdAt and updatedAt. Server timestamps are authoritative. Cross-app enums come from packages/shared/types and schemas.

Security, Role Guards and Privileged Command Boundaries

UI guards improve UX; Firestore rules and Cloud Functions enforce security. Never rely on hidden buttons as authorization.

1

Role and scope matrix

Capability	Customer	Merchant	Driver	Admin	Super Admin
Browse stores/items	Y	Own store preview	Y	Y	Y
Place / track order	Y	Read own store	Assigned	Read all scoped	Read all
Update order status	Cancel/request	Confirm/prepare/ready	Accept/on-way/delivered	Override / resolve	Override / resolve
Manage stores/items	-	Own store scoped	-	Y	Y
Assign driver / refund	-	-	-	Y	Y
Manage users / roles	Own profile	Own profile	Own profile	Suspend scoped	Full roles
Settings / fee rules	-	-	-	Y	Y

Scope invariant

Merchant reads/writes only assigned storelds. Driver reads assigned orders only. Customer reads own orders only. Admin access is explicit and audited.

3

Sensitive operations

Role assignment

server-side custom claim / role document; super-admin restriction

Store deletion

deleteStoreAdmin-style command; preserve orders; clean dependent references

Account deletion

disable Auth + archive/redact; retention-safe cleanup

Refund

validate payment state, amount, provider response and idempotency

Order override

allowed transition matrix + reason + audit

Import

sanitize URLs/CSV, restrict sizes, preview before publish

2

Defense in depth

1. Route guard

Block unauthenticated or wrong-role navigation.

2. UI capability guard

Hide / disable controls outside role and scope.

3. Shared schema validation

Validate payload shape and transition intent.

4. Firestore rules

Deny by default; enforce ownership and immutable fields.

5. Cloud Functions

Privileged delete, role, refund, webhook and cleanup commands.

6. Provider verification

Verify payment webhook signature and event replay.

7. Audit and alert

Record actor, target, result and suspicious failures.

Monorepo Architecture and Dependency Boundaries

Integrate the five experiences incrementally without creating a second Firebase project or duplicating collections.

1

Target workspace

projectweb/

apps/

- admin-web/ React + TypeScript
- customer-web/ React web
- customer-app/ Expo + React Native
- driver-app/ Expo + React Native
- merchant-web/ React + TypeScript

packages/

- shared/
- firebase/
- types/
- schemas/
- services/
- utils/
- config/ lint, tsconfig, env validation

firebase/ rules, indexes, functions

scripts/ checks and migration utilities

UI boundary

Do not share admin tables/modals with Expo apps. Share domain logic, Firebase adapters, schemas, status helpers and pure calculations.

2

Non-negotiable migration constraints

- 1

Keep one Firebase project and the existing collections.
- 2

Preserve customer-app and driver-app EAS project IDs/configuration.
- 3

Do not delete the old apkproject until the customer app works inside the monorepo.
- 4

Inspect both folders and create a migration plan before moving files.
- 5

Move shared code only where dependency direction is clear and tests pass.
- 6

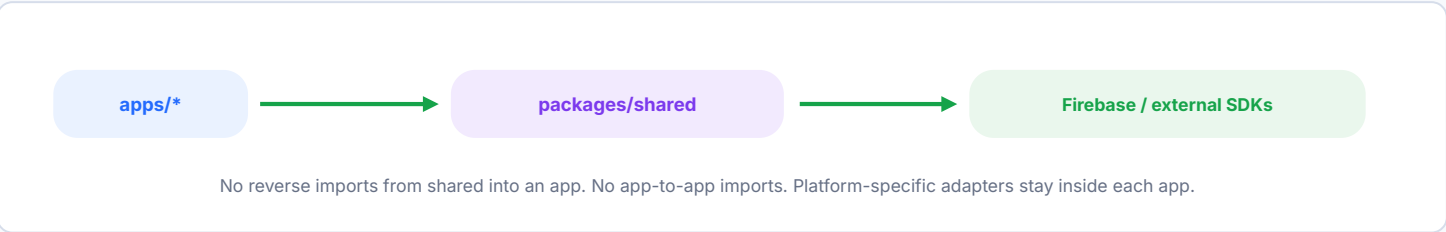
Keep Spaceman branding and existing working behavior during migration.
- 7

Report every moved, created and modified file.
- 8

Run typecheck and build for each app after every bounded phase.

3

Dependency direction



Performance, Reliability and Operational Observability

Optimization focuses on listener discipline, pagination, image/map cost, idempotent commands and explicit test coverage.

1 Performance guardrails



Realtime listeners

Only active screens and active orders. Unsubscribe on route/tab change; share duplicate subscriptions.



Long lists

Server pagination; 50 item target for catalog; virtualized rows where needed.



Images

Compress before upload, generate thumbnails, lazy-load and clean orphaned media.



Maps

Lazy-load map panels; throttle driver locations; cache route/ETA within freshness window.



Queries

Use indexed filters, bounded date ranges and denormalized display snapshots.



Bundles

Route-level code splitting for web; avoid importing admin-only libraries into customer bundles.



Caching

Cache stores/categories/catalog; invalidate after admin writes; do not cache payment truth.

2 Reliability patterns

Idempotency

Imports, webhooks, refunds and destructive commands use stable request/event IDs.

Transactions

Use Firestore transactions/batches for coupled status and audit writes.

Optimistic UI

Only for reversible low-risk edits; not for paid/refunded/delivered truth.

Retry policy

Exponential backoff for transient provider failures; surface final failure as activity.

Schema versioning

Readers tolerate previous versions; migrations are explicit scripts.

Fallbacks

Maps/payment failure never silently fabricates ETA or payment state.

Cleanup

Scheduled checks for stale locations, orphaned files and stuck imports.

3 Observability and test matrix

Audit logs

role changes, deletes, refunds, overrides, fee changes

Activities

human work queue for errors and requests

Function logs

command ID, actor, target, duration, result

Notifications

queued, sent, delivered/read when available

Tests

unit schemas/formula, emulator rules/functions, web E2E, mobile smoke

Release gate

typecheck + lint + build for every app, Firestore rules/index validation

Avoid known slowdowns

Too many Firestore listeners, uncompressed images, heavy localization, constantly rendered maps, unvirtualized lists and admin-style tables in mobile apps.

Codex Implementation Contract and Prompt Checklist

Use this page at the start and end of each bounded implementation task to protect the monorepo and shared backend.

1

Required preflight

1

Read this architecture page relevant to the task.

2

Inspect current repo files and git status.

3

Summarize current behavior and completed work.

4

List exact files and data paths in scope.

5

Identify security rules, functions and schema impact.

6

Create an incremental plan before editing.

2

Change constraints

!

Do not restart or revert completed work.

!

Do not broaden scope or refactor unrelated modules.

!

Do not create a second Firebase project or duplicate collections.

!

Do not loosen Firestore rules to make a feature pass.

!

Preserve manual store/item methods and existing EAS IDs.

!

Use shared packages only when safe; avoid cross-app UI imports.

3

Verification and report

OK

Run targeted tests first.

OK

Run typecheck / lint.

OK

Build affected app(s).

OK

Validate Firebase rules/functions/indexes if touched.

OK

List every changed file.

OK

Report schema and deployment changes separately.

OK

State remaining risks and manual tests.

4

Reusable bounded-task prompt

Resume from current progress. Do not restart, revert or refactor unrelated code.

Task: [one bounded feature from this architecture].
Scope: [app/module/files]. Preserve the five-app monorepo, one Firebase project and existing collections.

Before editing:
1. Summarize current implementation. 2. List files/data paths in scope. 3. State schema/rules/function impact.

Implement incrementally using shared types/schemas/services only where safe. Keep manual workflows working. Privileged commands must remain server-validated and audited.

After editing: run targeted tests, typecheck/lint and affected builds. Report every changed file, schema/deployment changes, manual tests and remaining risks.

Source precedence

1) Current repo and deployed backend truth. 2) This approved architecture and project decisions. 3) Source mockups for UI intent. When they conflict, stop and report the conflict rather than guessing.

Traceability, Normalizations and Architecture Decisions

This matrix connects the supplied boards to the synthesized system and records decisions that supersede early mockup assumptions.

1

Source-to-module traceability

Source board	Captured architecture detail	Synthesized in
Activities overview	Activities, fee changes, alerts, support, account requests	Page 10
Simple fee setup	Pricing formula, live preview, maps dependency	Page 10
Items workflow	Manual/API/CSV/Browse, preview and pagination	Page 6
Order data/status	Canonical order, ownership, lifecycle and reason codes	Pages 8-9
Orders - Needs action	KPI/tabs/filter/table/action contract	Page 7
Orders - Unassigned	Waiting-time and assign-driver view	Page 7
Orders - All visual	Status badges and table visual language	Pages 3 and 7
Stores workflow	Google/manual/merchant create, edit and delete	Page 5
Dashboard workflow	Navigation, counts, role/account modals	Page 4

Document status

Architecture synthesis for implementation. Validate exact existing file paths and non-order collection names against the monorepo before code changes.

2

Normalized labels and current decisions

Activities	Normalize earlier "Actives" nav label to Activities.
Account deletion	Archive/redact + Auth disable; preserve orders/audit.
User management	Suspend / unsuspend for temporary access control.
Store/item imports	Optional accelerators; manual creation remains.
Pricing v1	Mabopane single zone, standard delivery, no surge.
Backend	One Firebase project and one shared data model.
UI stacks	Web remains React; mobile remains Expo React Native.
Delete operations	Privileged Functions; preserve historical order integrity.

Use during review

For a Codex change, point to the relevant page number, state which source board it traces to, and list any approved decision that supersedes the mockup.

Activities and Fee Configuration Source Board

Activities overview, fee configuration and account requests

Dashboard

Stores

Items

Orders

Activities

Analytics

Settings

Admin
Super Admin

Activities

Monitor and manage key platform activities

Mabopane, Pretoria

May 19 - May 26, 2025

4

Pending

2

This week

3

Needs attention

7

Open

All Activities

Delete Account Requests

Delivery Fees

System Alerts

Support Tickets

Activity Type	Description	Related To	Requested By	Date & Time	Status	Action
Delete Account Request	Customer requested to delete account and all data	User: Mpho M. mm***@gmail.com	Customer (App)	May 26, 2025 10:45 AM	Pending	Review
Delivery Fee Change	Delivery fee updated for Mabopane zone	Mabopane Zone	Admin	May 26, 2025 09:30 AM	Approved	
System Alert	Driver location not updated for order #10451	Order #10451	System	May 26, 2025 09:10 AM	Open	
Support Ticket	Customer reported late delivery	Order #10448	Customer (Web)	May 25, 2025 08:15 PM	Open	
Delete Account Request	Customer requested to delete account and all data	User: Lerato K. le***@gmail.com	Customer (Web)	May 25, 2025 05:40 PM	Approved	

Showing 1 to 5 of 16 activities

ACTIVITIES WORKFLOW OVERVIEW

Request / Event Triggered

From App, Web, System or Admin

Appears in Activities

Categorized by type

Admin Reviews Request / Event

Checks details & impact

Admin Takes Action

Approve, Reject or Resolve

System Updates

Status updated & user notified

ACTIVITY TYPES EXPLAINED

Delete Account Requests

Customers request to permanently delete their account and all related data.

Delivery Fee Changes

Admin updates delivery fee settings for the Mabopane zone.

System Alerts

Important system events that need admin attention (driver offline, order issues, etc.)

Support Tickets / Complaints

Customer or store raises a support request or complaint.

DELIVERY FEE CONFIGURATION (SIMPLIFIED FOR MABOPANE)

One local delivery zone – Mabopane, Pretoria

1. Set Base Delivery Fee

Base Fee (Flat Rate)

R 25.00

This is the minimum delivery fee for all orders in Mabopane.

2. Set Distance Fee (Optional)

Per additional km after base distance

R 2.00

Applies after the base distance. Helps cover longer trips.

3. Free Delivery Threshold (Optional)

Min. order amount for free delivery

R 150.00

Orders equal or above this amount get free delivery.

Current Delivery Fee Preview

Example Calculation (5 km trip, order R100)

Base Fee R25.00

Distance (3 km x R2.00) + R6.00

Total Delivery Fee R31.00

* Within Mabopane zone only

Zone Settings

Delivery Zone

Mabopane, Pretoria

Zone Status

Active

Save Changes

Simple & Local

Only one zone (Mabopane) for now. Prices apply to all stores and customers in this area.

Recent Changes

May 26, 2025 09:30 AM Base fee changed to R25.00 Admin

May 20, 2025 11:15 AM Distance fee changed to R2.00 Admin

View all changes

This simple model is industry standard for local delivery startups. You can expand to multi-zone & dynamic pricing later.

DELETE ACCOUNT REQUESTS

Customers request to permanently delete their account from App or Web

4

Pending

2

Approved

8

This Month

When approved, user account and all related data is permanently removed from the system.

User	Source	Requested On	Reason (Optional)	Status	Action
Mpho M. mm***@gmail.com	App	May 26, 2025 10:45 AM	No longer using the app	Pending	Review Reject
Lerato K. le***@gmail.com	Web	May 25, 2025 05:40 PM	Privacy concerns	Pending	Review Reject
John D. jo***@gmail.com	App	May 24, 2025 02:18 PM	Found another service	Approved	—
Neo M. ne***@gmail.com	Web	May 20, 2025 11:22 AM	No reason provided	Approved	—

After approval: All user data (profile, addresses, orders, cart, favorites, notifications, login info) is permanently deleted and cannot be recovered.

1 Extracted Codex cues

1

Sidebar: Dashboard, Stores, Items, Orders, Activities, Analytics, Settings.

2

Activity cards: deletion requests, fee changes, system alerts, support tickets.

3

Workflow: trigger → categorize → review → action → update/notify.

4

Fee UI includes base, distance, threshold, preview, zone state and audit history.

5

Deletion mockup is superseded by archive/redact architecture decision.

Traceability

This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.

SPACEMAN PROJECTS - ADMIN WEB ARCHITECTURE

Implementation reference for Codex / VS Code

1

Spaceman Activities — Simple Delivery Fee Setup

1 SIMPLE DELIVERY FEE CONFIGURATION LAYOUT

 Save Rule

R32.80

Implementation reference for Codex / VS Code	18
--	----

Spaceman Admin — Items Workflow

1 PAGE OVERVIEW / DEFAULT STATE

Dashboard
Stores
Items
Orders
Actives
Analytics

Add Item Method

Manually API CSV Browse

Search store

Search store (inactive)

Browse allows edits.

By default, the Search store input is inactive until one of the method buttons is selected.

2 METHOD A: MANUALLY

1. Select "Manually"

2. Activate store search

3. Search & select store

4. Display item creation form

Item name
Price (Rands)
Category
Description

Item image preview

Upload Delete

Create

Create adds the item to Firebase.

3 METHOD B: API IMPORT

1. Select "API"

2. Activate editable store search

3. Search & select store

4. Enter API link & preview

5. Preview import candidates

500 items Select all

Item name 1 R10

Item name 2 R20

Item name 3 R30

Add Items

- Preview displays import candidates.
- Select items to import.
- Items may include name, price, category, description, image, metadata if available.
- Preview display focuses on image, name, and price.
- After preview, use Add Items to add only selected items to Firebase.

4 METHOD C: CSV IMPORT

1. Select "CSV"

2. Activate editable store search

3. Search & select store

4. Upload CSV

5. Preview uploaded items (sample)

Image Item name Price (Rands) Select

Item name 1 R10

Item name 2 R20

Item name 3 R30

- CSV contains: name, price, category, description, image name.
- CSV should include actual image files corresponding to the image names for orderly upload.
- Display preview after upload.
- Show only a small sample for preview, then full list.
- Pagination: 50 items per page.
- After preview, replace Preview with Add Items.
- Add Items adds only selected items to Firebase.
- Each item should contain: name, price, category, description, image, metadata if available.
- Preview display focuses on image, name, and price.

5 METHOD D: BROWSE / MANAGE EXISTING ITEMS

1. Select "Browse"

2. Activate store search

3. Search & select store

4. Display all items of selected store (replaces store search with item search)

Store one

Image Item name Edit Delete

Item name 1

Item name 2

Item name 3

5. Edit item (opens module)

Item name
Price (Rands)
Category
Description

Item image preview

Upload Delete image

Update

Notes

- Items are paginated: 50 items per page.
- Search item filters items of the selected store.
- Each row contains image, name, edit, and delete.
- Edit opens populated item fields in a module.
- Update button below the fields updates the item data on Firebase.
- Delete button on each item row permanently deletes the item on Firebase.

6 COMMON SYSTEM RULES

One store must be selected before creating or importing items.

Store name is locked in methods that target a selected store.

Import previews are for review before committing to Firebase.

Browse mode is for searching, editing, and deleting existing items.

Imports and browse lists use pagination, typically 50 items per page.

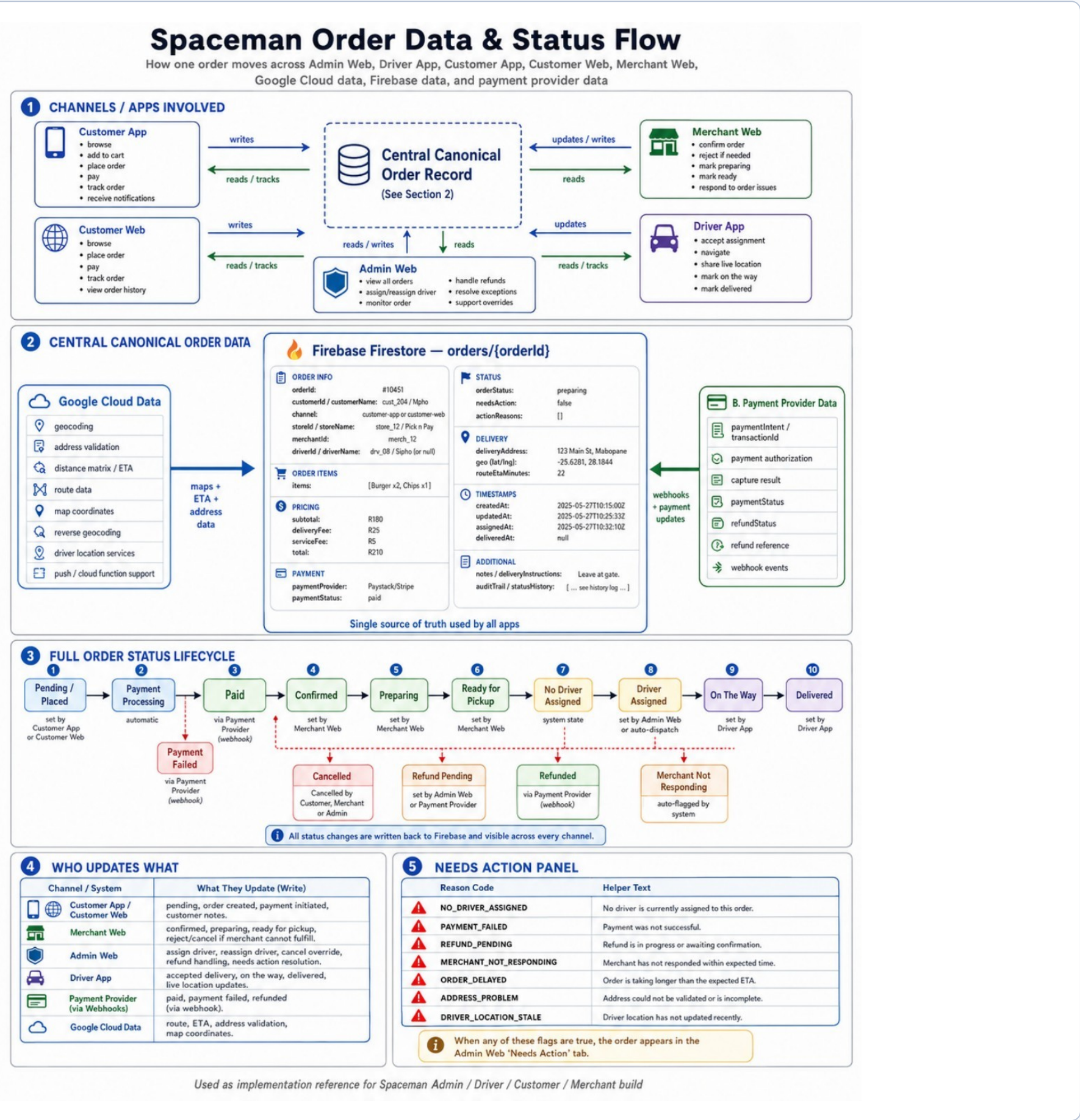
Firebase is the final data destination for create, import, update, and delete actions.

Traceability

This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.

Order Data and Status Source Board

Canonical order data and status lifecycle



Orders - Needs Action Source Board

Orders console - Needs action tab

Orders

🔄 Live refresh

📄 Export

⌵ Filters

🛒

New

12

⚠️

Needs action

8

👤

Unassigned

5

🍽️

Preparing

18

🚚

On route

27

🕒

Late

4

All

Needs action

Active

Unassigned

Late

Completed

🔍

Search order/customer/store/driver...

📅

Date

⌵

📍

Zone

⌵

⌵

Status

⌵

Order	Store	Customer	Payment	Status	Issue	Driver	Action
#10451	Pick n Pay	Mpho	Paid	Unassigned	No driver assigned	—	Assign driver
#10453	Amigos Cafe	Lerato	Payment failed	Pending	Payment failed	—	Review payment
#10454	Steers	John	Refunded	Cancelled	Refund pending	Kabelo	Process refund
#10455	Nando's	Thato	Paid	On route	Order delayed	Sipho	Contact driver

1 Extracted Codex cues

1

Stats: New, Needs action, Unassigned, Preparing, On route, Late.

2

Tabs and filter bar are stable across views.

3

Needs-action table adds Issue and row-specific Action.

4

Examples: assign driver, review payment, process refund, contact driver.

Traceability

This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.

SPACEMAN PROJECTS - ADMIN WEB ARCHITECTURE

Implementation reference for Codex / VS Code 21

Orders - Unassigned Source Board

Orders console - Unassigned tab

Orders

Live refresh

Export

Filters

New
12

Needs action
8

Unassigned
5

Preparing
18

On route
27

Late
4

All

Needs action

Active

Unassigned

Late

Completed

Search order/customer/store/driver...

Date

Zone

Status

Showing active orders with no driver assigned.

Order	Store	Customer	Payment	Status	Waiting time	Driver	Action
#10451	Pick n Pay	Mpho	Paid	Unassigned	18 min	—	Assign driver
#10456	Burger Hub	Neo	Paid	Confirmed	11 min	—	Assign driver
#10457	Amigos Cafe	Sarah	Paid	Preparing	9 min	—	Assign driver
#10458	Chicken Licken	Kabelo	Paid	Ready	6 min	—	Assign driver

1 Extracted Codex cues

- 1

Unassigned tab includes helper text and Waiting time.
- 2

Rows include active orders in Confirmed, Preparing or Ready states.
- 3

Primary row command is Assign driver.
- 4

Waiting time should derive from status timestamps.

Traceability

This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.

Orders - All Tab Visual Source Board

Orders console - All tab visual style

Orders

Live refresh

Export

Filters

New

12

Unassigned

5

Preparing

18

On route

27

Late

4

All

Needs action

Active

Unassigned

Late

Completed

Search order/customer/store/driver...

Date

Zone

Status

Order	Store	Customer	Payment	Status	Driver	ETA
#10452	Nando's	Thato	Paid	Preparing	Sipho	22m
#10451	Pick n Pay	Mpho	Paid	Unassigned	—	Late
#10450	Steers	John	Refunded	Cancelled	Kabelo	—

1 Extracted Codex cues

1 Colored KPI icons and text badges establish semantic visual language.

2 All table columns: Order, Store, Customer, Payment, Status, Driver, ETA.

3 Order IDs are links; payment/status use explicit text plus color.

4 Filter button is primary blue action.

Traceability
This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.

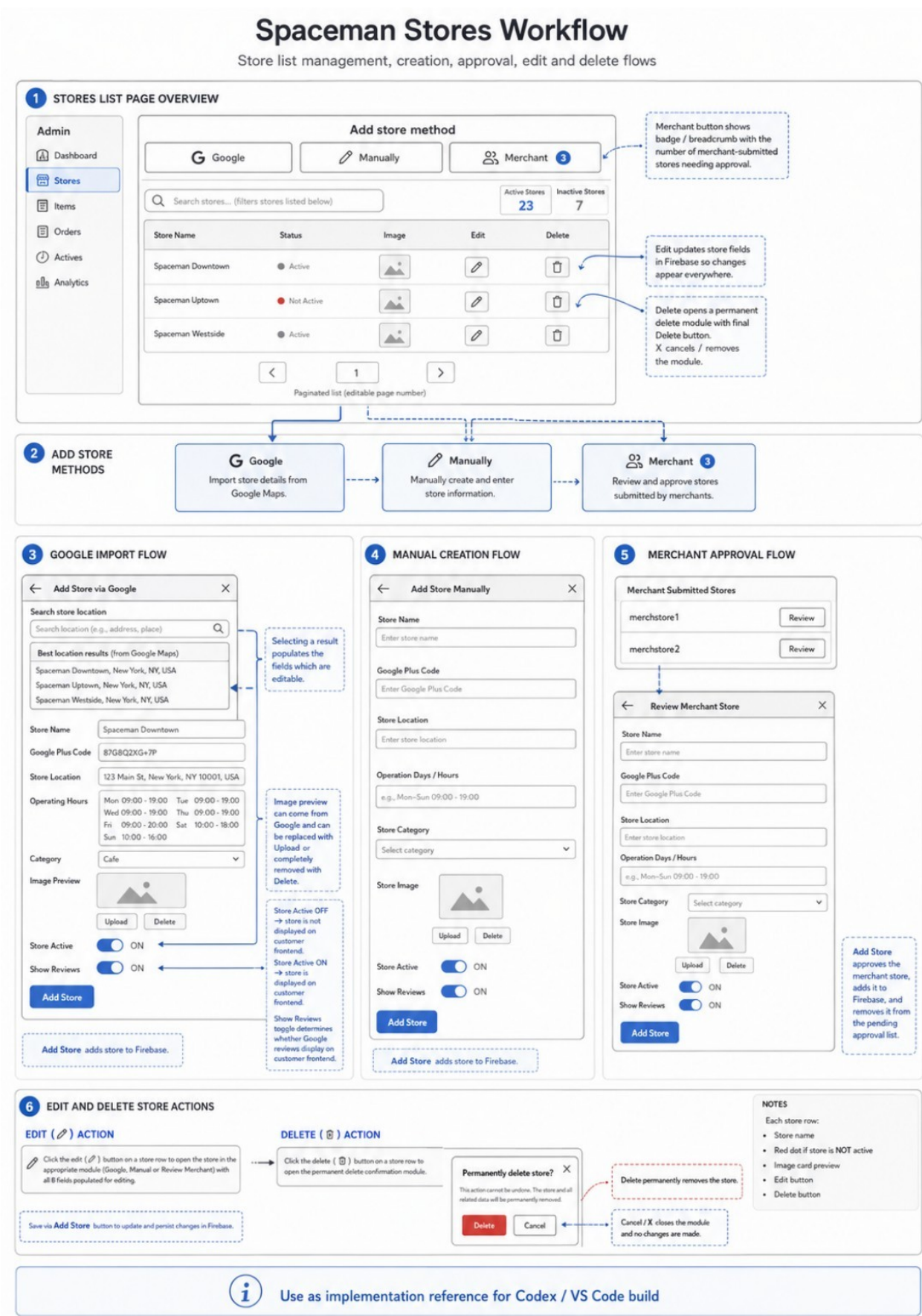
SPACEMAN PROJECTS - ADMIN WEB ARCHITECTURE

Implementation reference for Codex / VS Code

23

Stores Workflow Source Board

Stores workflow - Google, manual and merchant



1 Extracted Codex cues

1

List supports search, active/inactive counts, pagination, edit and delete.

2

Creation routes: Google, Manual, Merchant approval.

3

Google selection prefills editable store fields.

4

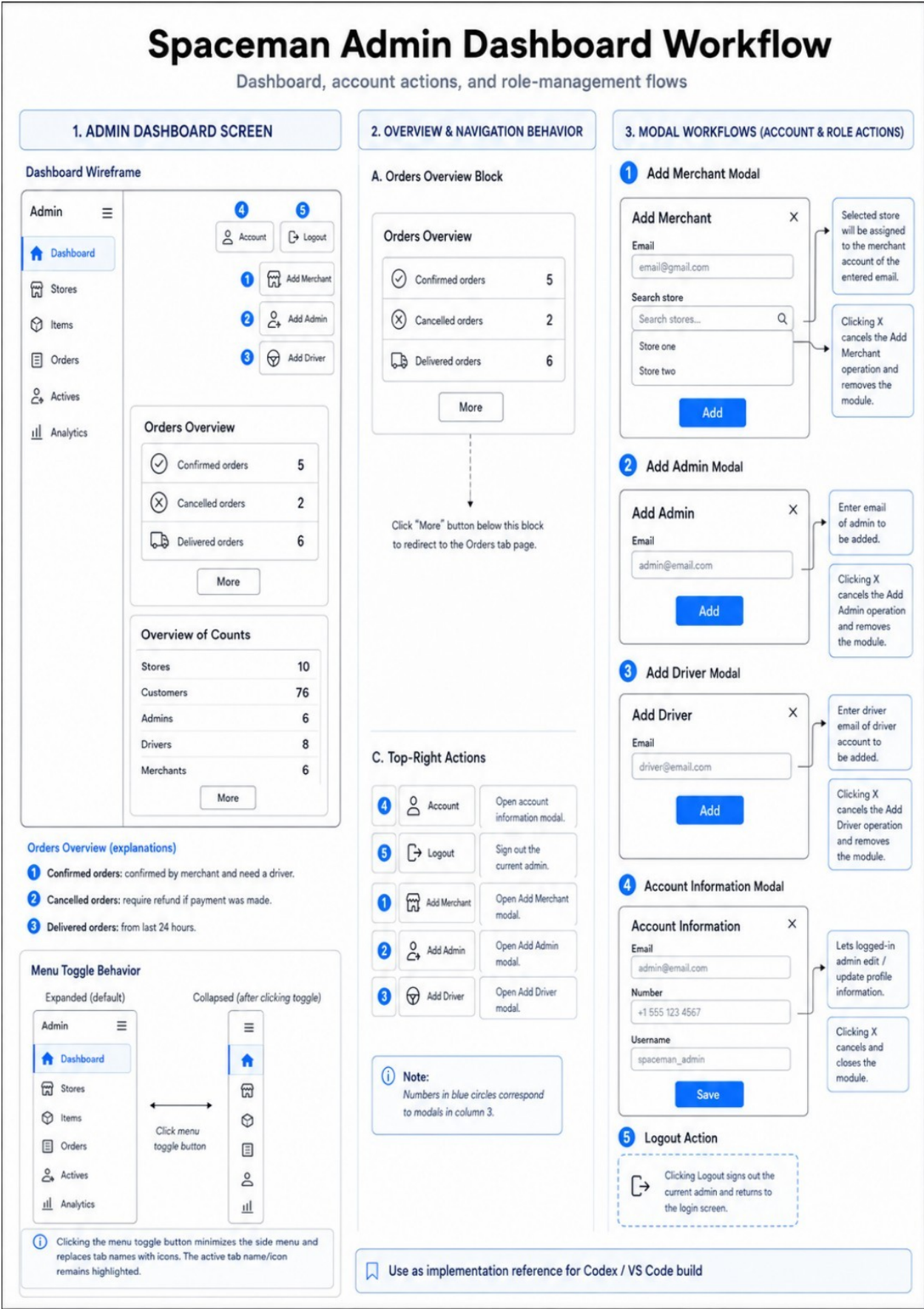
Merchant submissions enter an approval queue with badge count.

5

Edit writes shared data; delete requires confirmation.

Dashboard and Role Modals Source Board

Dashboard, navigation and role-management modals



1 Extracted Codex cues

1

Dashboard includes orders overview and platform counts.

2

Sidebar collapses from labels to icons while active state remains.

3

Top-right actions: Add Merchant, Add Admin, Add Driver, Account, Logout.

4

Merchant creation requires email and store assignment.

5

Account modal edits profile fields, not authorization claims.

Traceability

This page preserves the original visual reference. Use the synthesized architecture pages for implementation decisions and this board for exact UI intent.